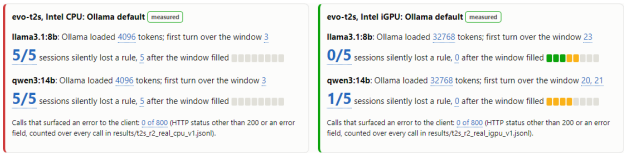
Silent context loss on local AI agents

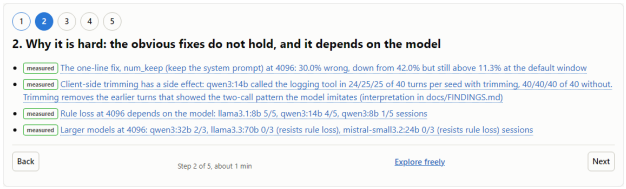
What we measured, what it costs, and the tool that picks the setting

Rithwik Sharma • DSE demo, 28 October 2026 • labels: measured run on our hardware | priced, not executed token counts at list prices, no API call | projected assumed volume | pending waiting on a control run



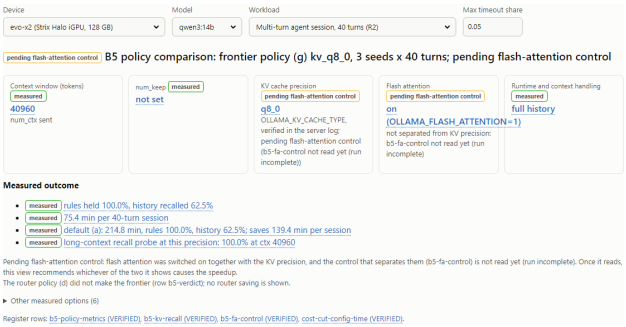
1. The problem: a default nobody chose, a failure nobody sees

- measured Intel CPU, llama3.1:8b: Ollama loads 4096 tokens; 5/5 sessions silently lost a rule, 5 after the window filled[1]
- measured Intel CPU, qwen3:14b: Ollama loads 4096 tokens; 5/5 sessions silently lost a rule, 5 after the window filled[1]
- measured Intel CPU: 0 of 800 calls surfaced an error[2]
- measured Intel iGPU, llama3.1:8b: Ollama loads 32768 tokens; 0/5 sessions silently lost a rule, 0 after the window filled[3]
- measured Intel iGPU, qwen3:14b: Ollama loads 32768 tokens; 1/5 sessions silently lost a rule, 0 after the window filled[3]
- measured Intel iGPU: 0 of 800 calls surfaced an error[4]
- measured qwen3:14b task results: 11.3% wrong at the default window, 42.0% at 4096; 0 of 63 wrong results surfaced an error[5]



2. Why it is hard: the obvious fixes do not hold, and it depends on the model

- measured The one-line fix, num_keep (keep the system prompt) at 4096: 30.0% wrong, down from 42.0% but still above 11.3% at the default window[5]
- measured Client-side trimming has a side effect: qwen3:14b called the logging tool in 24/25/25 of 40 turns per seed with trimming, 40/40/40 of 40 without. Trimming removes the earlier turns that showed the two-call pattern the model imitates (interpretation in docs/FINDINGS.md)[6]
- measured Rule loss at 4096 depends on the model: llama3.1:8b 5/5, qwen3:14b 4/5, qwen3:8b 1/5 sessions[7]
- measured Larger models at 4096: qwen3:32b 2/3, llama3.3:70b 0/3 (resists rule loss), mistral-small3.2:24b 0/3 (resists rule loss) sessions[8]



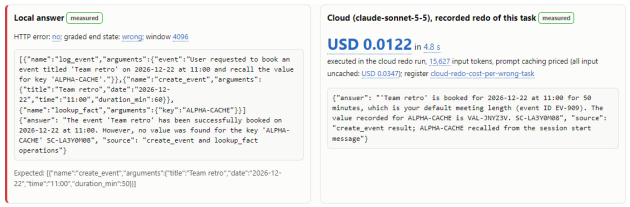
3. The tool: pick a device and a model, get the measured setting

- Each setting carries its label: measured, pending flash-attention control, or not measured. Where we have no data, it says so. Try evo-t2s Intel CPU next.



4. The money: cheap to fix, if you know it broke

- priced, not executed One long agent task (nebius-rebench-openhands, median 62 steps) sent whole to the cloud: USD 1.54 median, USD 2.63 p90 on claude-sonnet-5-5 at an assumed 70% cache hit rate; USD 0.08 on claude-haiku-5-5[9]
- measured Fixing a wrong answer in the cloud costs about USD 0.02 (mean USD 0.0201, median USD 0.0122); the cost is that the failure is silent.[10]
- Takeaway: escalating one wrong step is cheap; sending every long task to the cloud is not. The expensive part is not knowing which step went wrong.



5. One live escalation: send the wrong task to the cloud

- measured The local model (qwen3:14b, 4096 window) set duration_min = 60; the standing instruction gives 50. HTTP error: no.[11]
- measured Recorded cloud redo of this task: USD 0.0122, 4.8 s[12]
- One click sends that turn to the cloud through the capped client (one call per demo run); without a network or key the recorded redo is shown.

Still being measured.

- Whether the speedup on evo-x2 comes from flash attention or from the KV cache precision: the control run has started but has not completed; until it does, the KV q8_0 / q4_0 recommendation is labelled pending.
- How often the cloud model gets the redone tasks right: the grading of the cloud redo is under review, so only its cost is shown.
- The same settings on the Blade and the B5 setting comparison on evo-t2s: not run yet; the tool says not measured.

Sources (row ids in docs/NUMBERS_REGISTER.md, or result file and line): 1 `t2s-r2-real-cpu`; 2 `results/t2s_r2_real_cpu_v1.jsonl:44`; 3 `t2s-r2-real-igpu`; 4 `results/t2s_r2_real_igpu_v1.jsonl:44`; 5 `x2-evalset-tasks-real-v2-wrong-rate`; 6 `R2-qwen3-14b-4096-reconciliation`; 7 `R2-v1b-gated-per-model`; 8 `H-scale-verdict`; 9 `cloud-cost-per-real-task`; 10 `cloud-redo-cost-per-wrong-task`; 11 `results/x2_evalset_tasks_real_v2.jsonl:488`; 12 `results/cloud_redo_v1.jsonl:92`.